HW3:

Motion Estimation for Video Coding

ABLE OF CONTENTS

LIST OF FIGURES

1.0 Introduction

Motion Picture Experts Group (MPEG) is a group formed in 1988. They are part of the International Organization of Standardization (ISO) that is responsible for developing standards. MPEG Group is responsible for developing an international compression standard for audio and video compression and transmission. There have been many versions of the MPEG standards with the latest version of the standards being the H.264 Compression Standards. The inception of these standards marked a leap in telecommunications industry as other protocols could be developed for efficient transmission of video and audio over wireless networks.

In the Signal Processing and Multimedia industry this is a very popular area of research. Much of the emphasis in this area is laid on finding Faster and more efficient ways to calculate Motion Vectors and to find optimal Block Sizes. For example, there are various Motion Compensation Algorithms. Two of which are investigated in this project, namely Exhaustive Search Algorithm and Conjugate Directional Search Algorithm.

The report is organized as ollows:

2.0 PROCEDURE FOR EXHAUSTIVE SEARCH

2.1 AIM

The aim of this project is to perform Motion Compensation for the coast guard video sequence. Two proposed algorithms are used. Firstly, the Exhaustive Search Algorithm is explored. The details of the procedure are outlined below.

2.2 STEPS UNDERTAKEN

2.2.1 Read the sequence

The first step to any Image Processing related algorithm is to first load the images or video sequences into the workspace so that it can be manipulated. It should be noted that since Matlab does not read YUV files. Instead, these files should first be converted to .avi files. The video sequence used for this project is Walk_qcif.avi. It should be noted that a qcif sequence has a frame size of 144x176. Once the file is read, the Reference and Target frames need to be individually extracted and stored as the search algorithms work with 2 frames at a time.

2.2.2. Extract Block from the Target Frame

Once the Reference and Target frames are set then, the next step would be to extract an individual Macroblock. A function was written to do this. This function takes in x and y coordinates in increments of 16 and returns a single block. The entire code is written for a single Macroblock and iterations step from one Macroblock to another.

2.2.3. Extract Search Window from Reference Frame

Once the Macroblock is extracted, the next step is to extract a search window for the Macroblock. However, the Search Window is extracted from the Reference Frame while the Macroblock comes from the Target Frame. Before presenting a synopsis of Search Window extraction, one should keep in mind the nature of the Search Window. The basic concept of Search Window comes from the fact that the best match for a macroblock can be enclosed in a square window with the Macroblock being equidistant from all corners of the Search Window. However, this is not possible for all the blocks as some blocks are in the 4 corners of the image or are either in the first and last rows or first and last columns. Consequently, different window sizes were defined. For exhaustive search, every possible displacement within a rectangular search window is attempted. The displacement that produces the minimum distortion is chosen as the motion vector.

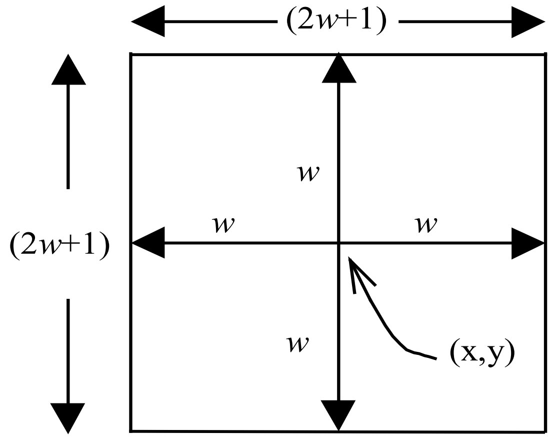

Figure 1.0: Shows the SW for ES

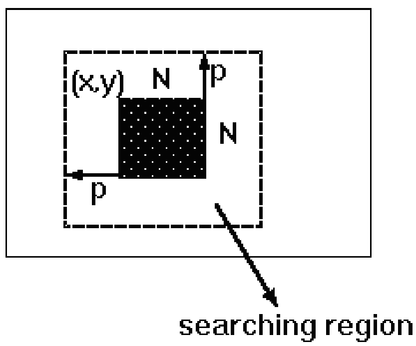

Figure 2.0: Shows an N x N Seach Window.

It should be noted that for a given block, if the Macroblock cannot be placed equidistant from the edges of the Search Window, then the Macroblock should be placed 8 pixels away in each allowable direction. Thus, the combinations of the Search Window Dimensions are as follows: **30 x 30; 22 x 22; 30 x 22 and 22 x 30**.

2.2.4. Determine the Best Match Macroblock

The Exhaustive Search Algorithm Search is a block-based search algorithm. The Motion Estimation over a Macroblock of pixels is a standard approach for estimating motion in a moving image sequence. In this algorithm, every possible displacement within a rectangular search window is attempted. The displacement that produces the minimum distortion is chosen as the motion vector. To explain this further,

- Consider a single Macroblock.
- Consider the appropriate Search Window.
- Align the Macroblock on the top left side of the Window.
- Consider the first 16 x 16 pixels in the Window.
- Compute the pixel-by-pixel difference for that Macroblock.
- Then calculate the absolute sum of the Difference Matrix and store it into an array.
- Increment the column index of the Search Window by one pixel to the right and repeat step 4 – 6.
- Continue to increase the column index till the Macroblock does not exceed the column dimension of the matrix.
- Once, the column index has reached its dimension, then reset the column vector and increment to the next row
- Repeat steps 4-9 again till there are no more possible iterations
- This procedure is called a raster scan for the window.
- Then Choose the smallest Maximum Absolute Difference value as the point of best match.
- Then tabulate the coefficients for the new position of the Macroblock and store the Error Block.
- Pass new position coordinate values and store it for the Motion Compensation.

2.2.5. Mean Absolute Difference (MAD)

There are two popular ways to tabulate the distortion of Macroblock. These are Mean Square Error (MSE) and MAD. For Macroblock 16 x 16, by using MSE there will be 256 Multiplications and 768 Additions, whereas MAD uses 512 Additions. This is a 33.3% reduction in the number of additions performed. Furthermore, MAD performs just as well as MSE in terms of reducing the entropy. Hence, MAD is used. It is given by the formula below:

$$Distortion(B_k(n),d) \quad = \quad \sum_{\forall(x,y)\in B_k(n)} (U_k(x,y) - U_r(x+d_x, y+d_y))^2 \qquad \textbf{(1)}$$

For Mean Absolute Difference (MAD), the distortion for a macroblock $B_k(n)$ is given by:

$$Distortion(B_k(n),d) \quad = \quad \sum_{\forall(x,y)\in B_k(n)} \left| U_k(x,y) - U_r(x+d_x, y+d_y) \right| \qquad \textbf{(2)}$$

Where, for both cases, $U_k(x,y)$ represents pixels in the current frame, and $U_r(x,y)$ represents pixels in the reference frame.

2.2.6. Calculate and store MV coordinates

This function utilizes the positions of the Matched Macroblock and calculates the displacement of pixels. The results are then stored into a dynamic array that increments and stores as each block is dealt with. Once all the vectors are tabulated, the Motion Vector Image is displayed. Each dot indicates a Macroblock, and the arrows indicate the direction of motion of the Macroblock.

2.2.7. Reconstructed Image

Once the Motion Vectors have been generated, the Frame is reconstructed. Therefore, there is a decoder in every encoder. To decode the image, the Error Image that was generated from the Motion Vectors added pixel by pixel to the Reference Frame to retrieve the Target Frame. Subsequently, the Reconstructed Image is displayed.

3.0 RESULTS FOR EXHAUSTIVE SEARCH

FRAME 1

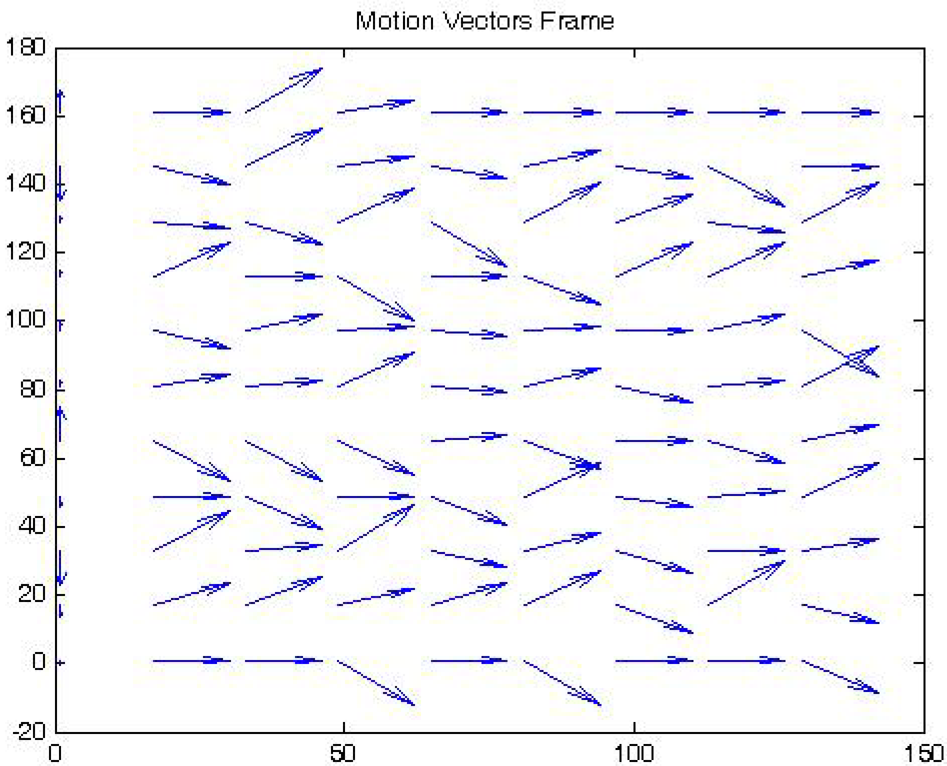

Figure 3.0: Motion Vector for Frame 1.

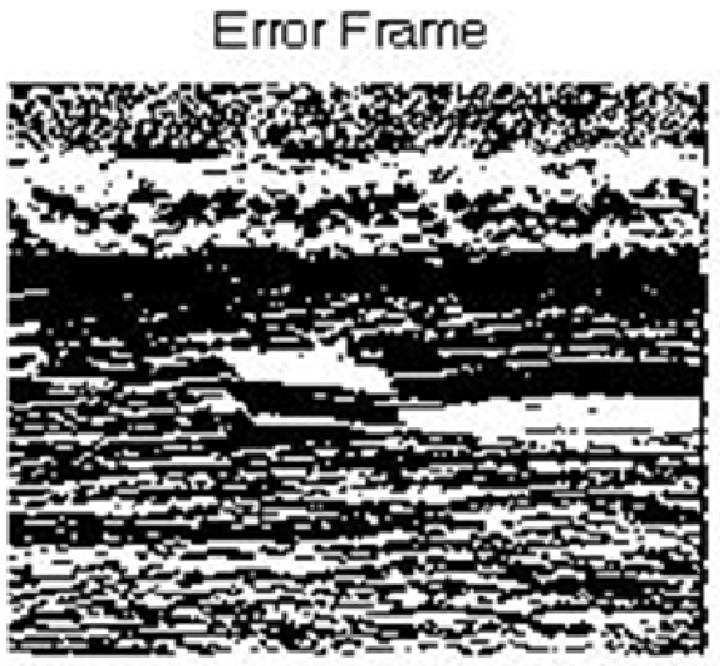

Figure 4.0: Shows the Error Frame 1.

Figure 5.0: Target Frame 1

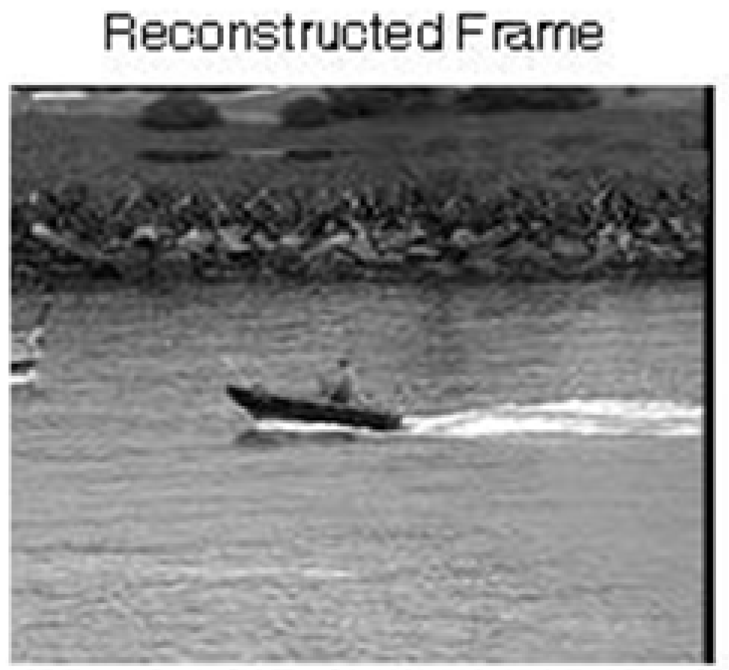

Figure 6.0: Shows the Reconstructed Frame 1

Similarly show other results for the REMAINING FRAMES.

4.0 Discussion of the results

5.0 Conclusions

8.0 References -cite any sources you have used.

1] Ze-Nian Li and Mark.S.Drew, "Fundamentals of Multimedia," Princeton Hall, 2004.

2] Website: [www.**wikipedia**.org](www.wikipedia.org)